

ENSF 380

TOPIC 15: COPYING OBJECTS

Object Reminders

Reminder: object = instance of a class

Objects are often named with nouns (e.g. Item)

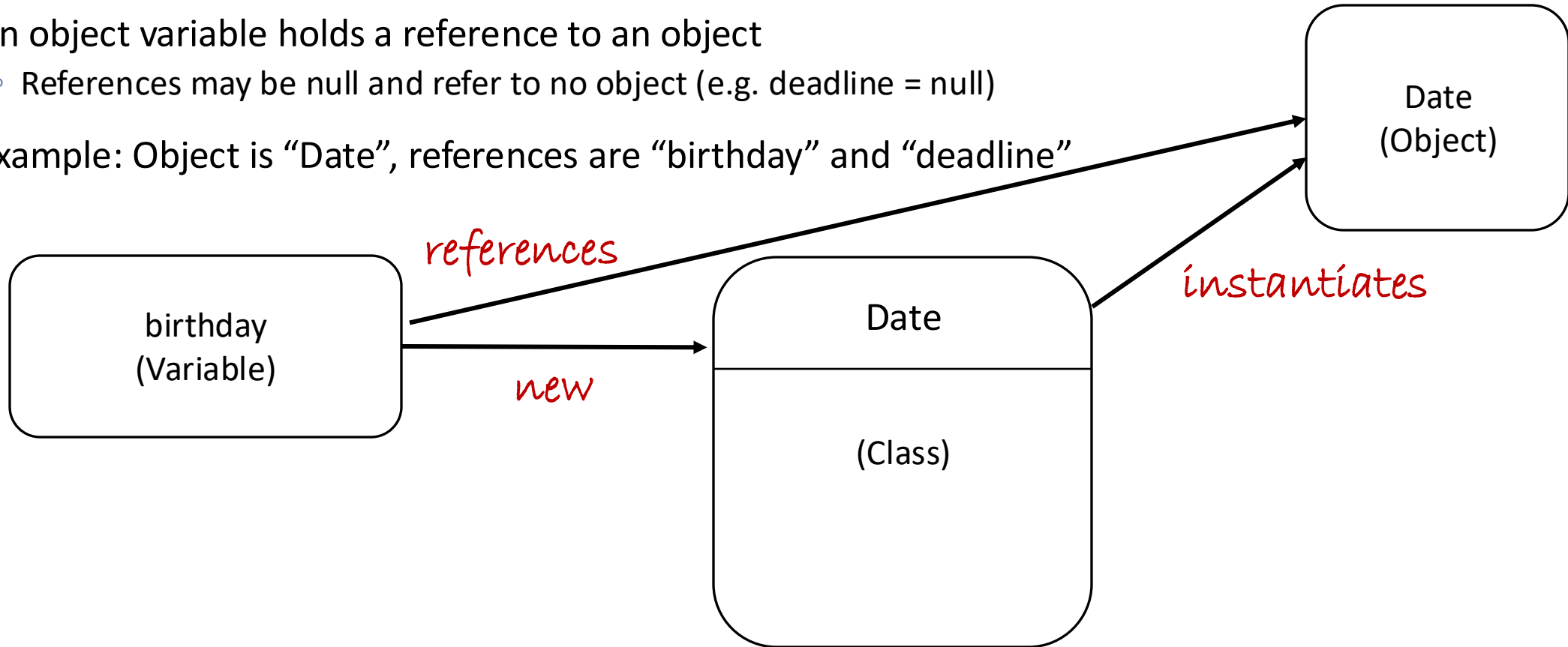
Objects have behaviours, often named with verbs (e.g. addItem, shipItem)

Object References

An object variable holds a reference to an object

- References may be null and refer to no object (e.g. deadline = null)

Example: Object is “Date”, references are “birthday” and “deadline”

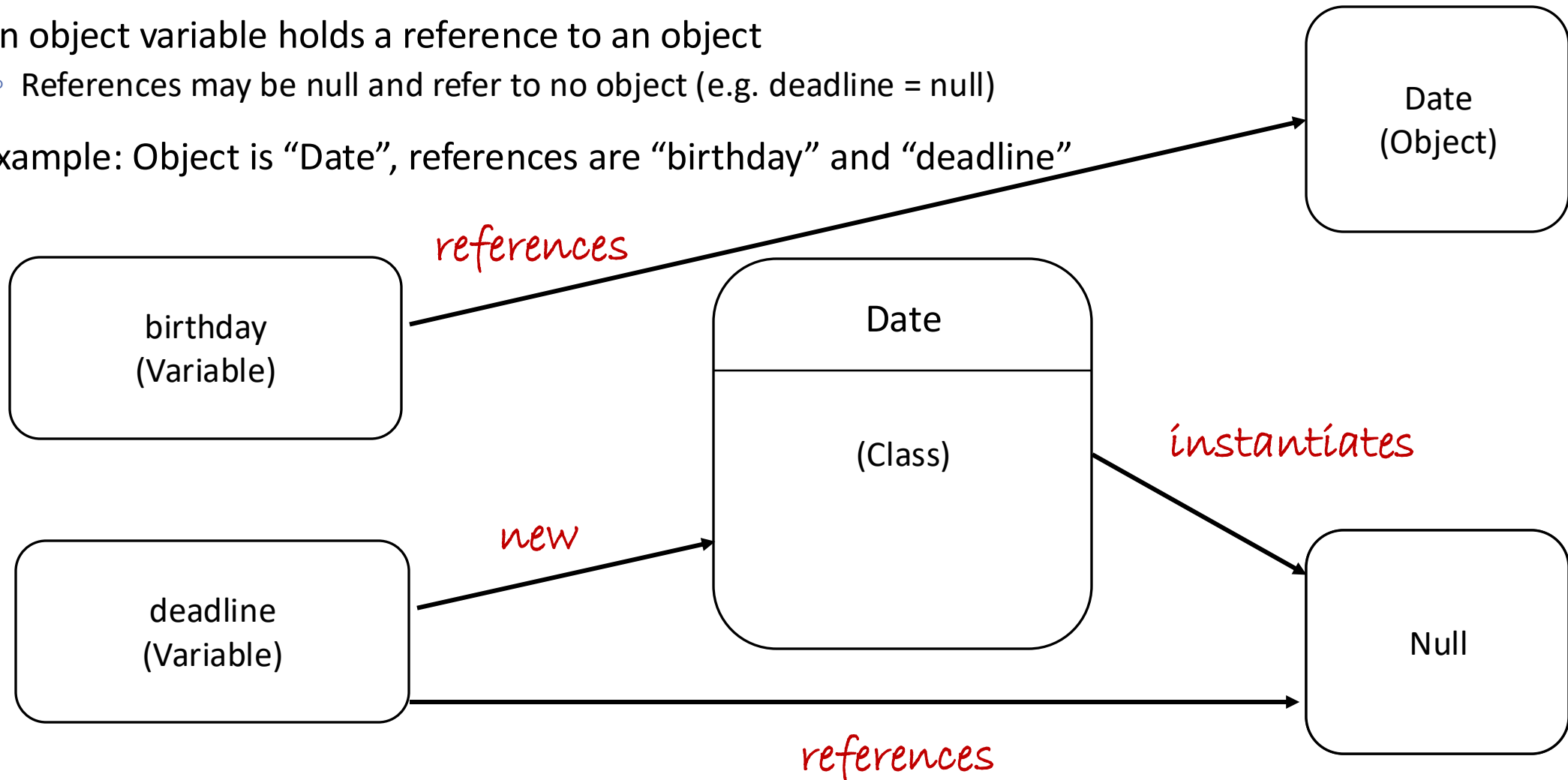


Object References

An object variable holds a reference to an object

- References may be null and refer to no object (e.g. deadline = null)

Example: Object is “Date”, references are “birthday” and “deadline”



Call by Value vs. Object References

Call by value: a method gets copies of argument values

- Methods do not change the contents of the variables that are passed in

Call with object references: allows methods to modify objects

In Java, everything is passed by value- including object references

Copying Objects in Java

Shallow Copy

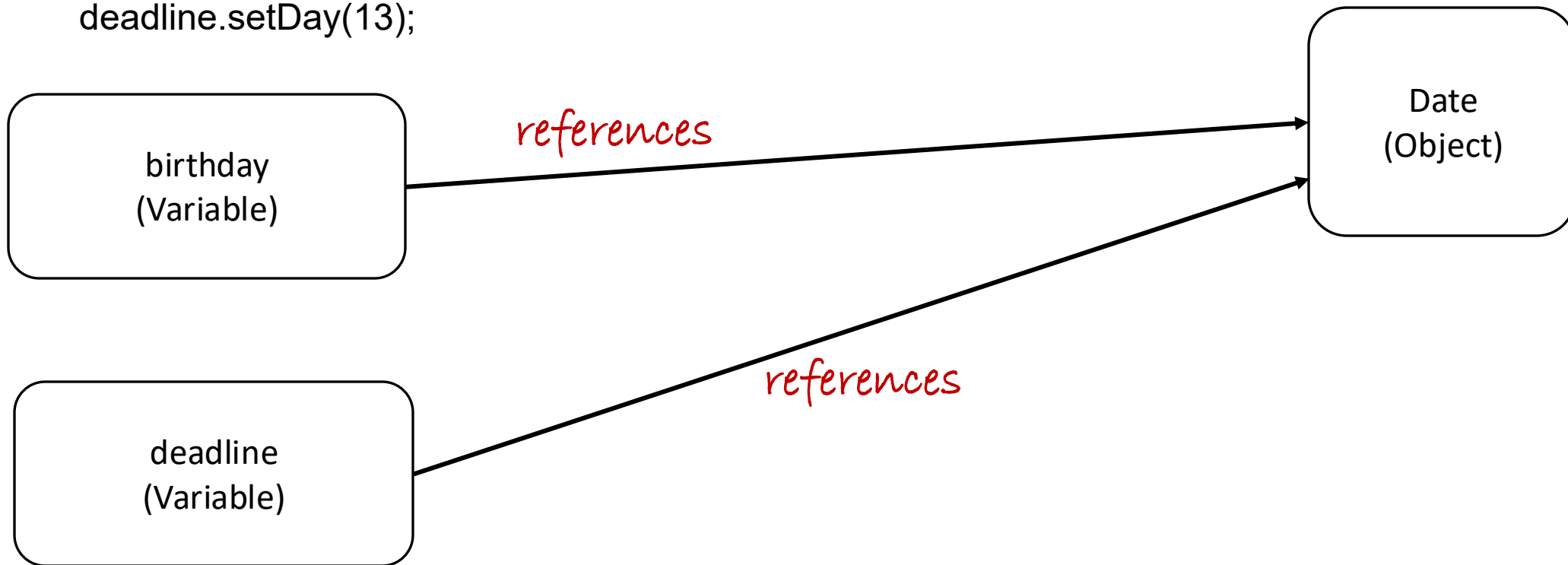
- Copies all primitive data type values
- Copies object reference types
- New reference variable will point to original
- Changing the new object will also change the original

Deep Copy

- Copies all primitive data type values
- Copies all non-primitive data type values
- Copy is independent of the original
- Costly operation

Shallow Copy

```
Date birthday = new Date(14 , 03 , 1986);  
Date deadline = birthday;  
  
deadline.setDay(13);
```



Object Copying Example: CompetitionHorse Class

01_SettingUp

CompetitionHorse class contains animal species, family, phylum, name, ten skills, and rider information

All setter and getter methods

```
...  
class CompetitionHorse{  
    private String animalSpecies;  
    private String animalFamily;  
    private String animalPhylum;  
    private String animalName;  
    private Rider animalRider = new Rider();  
    public String[] skills = new String[10];  
    ...  
}
```


Object Copying Example: Rider Class

01_SettingUp

Rider class contains rider name, their registration number, and their age

All setter and getter methods

```
...  
class Rider{  
    private String riderName;  
    private int riderRegNumber;  
    private int age;  
    ...
```

Create an Object

01_SettingUp

Create a horse and assign a rider. Our goal is to increase the herd through copying!

```
import java.util.Arrays;

public class ObjectCopying {

    public static void main(String[] args){

        CompetitionHorse horse1 = new
            CompetitionHorse("Horse","Equidae","Chordata","Blaze");
        String[] horse1Skills = new String[]{"Jumping", "Dressage"};
        horse1.setCompetitionHorseSkills(horse1Skills);
        horse1.setRider("Alex", 4321, 30);

        System.out.println("Name: " + horse1.getCompetitionHorseName());
        System.out.println("Skills: " + Arrays.toString(horse1.skills));
        System.out.println("Rider: " + horse1.getRider().getRiderName());
    }
}
```

```
Name: Blaze
Skills: [Jumping, Dressage]
Rider: Alex
```

Exercise 15.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Create a Shallow Copy

Increase the herd by copying the first horse.

```
...  
CompetitionHorse horse2 = new CompetitionHorse();  
horse2 = horse1;  
horse2.setCompetitionHorseName("Thunder");  
horse2.setRider("Taylor", 1234, 25);  
String[] horse2Skills = new String[]{"Racing"};  
horse2.setCompetitionHorseSkills(horse2Skills);  
...
```

Exercise 15.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Create a Shallow Copy

02_ShallowCopy

Increase the herd by copying the first horse.

```
...  
CompetitionHorse horse2 = new CompetitionHorse();  
horse2 = horse1;  
horse2.setCompetitionHorseName("Thunder");  
horse2.setRider("Taylor", 1234, 25);  
String[] horse2Skills = new String[]{"Racing"};  
horse2.setCompetitionHorseSkills(horse2Skills);  
...
```

```
Original Horse1  
Name: Blaze  
Skills: [Jumping, Dressage]  
Rider: Alex
```

```
Horse1  
Name: Thunder  
Skills: [Racing]  
Rider: Taylor  
Horse2  
Name: Thunder  
Skills: [Racing]  
Rider: Taylor
```

Clone Method

Implements the `Cloneable` interface (empty marker interface)

Need to `redefine the clone()` method

Handle `CloneNotSupportedException`

All objects contained within the cloned object must also be cloned!

Both Classes Implement Cloneable

03_DeepCopy

```
...  
class CompetitionHorse implements Cloneable{  
    private String animalSpecies;  
    private String animalFamily;  
    private String animalPhylum;  
    private String animalName;  
    private Rider animalRider = new Rider();  
    public String[] skills = new String[10];  
    ...
```

```
class Rider implements Cloneable{  
    private String riderName;  
    private int riderRegNumber;  
    private int age;  
    ...
```


Add CompetitionHorse Clone Method

03_DeepCopy

```
...  
public Object clone() throws CloneNotSupportedException {  
    CompetitionHorse copy = (CompetitionHorse)super.clone();  
    return copy;  
}  
...
```

Add Rider Clone Method

03_DeepCopy

```
... //...In CompetitionHorse Class
public Object clone() throws CloneNotSupportedException{

    CompetitionHorse copy = (CompetitionHorse)super.clone();
    copy.animalRider = (Rider)animalRider.clone();
    return copy;
}

... //...In Rider class
public Object clone() throws CloneNotSupportedException{

    return super.clone();
}

...
```

Add Exception to Main

03_DeepCopy

```
...  
import java.util.Arrays;  
public class ObjectCopying {  
    public static void main(String[] args) throws CloneNotSupportedException{  
...
```

Exercise 15.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Create a Deep Copy

03_DeepCopy

Increase the herd by cloning the first horse.

```
...  
CompetitionHorse horse2 = (CompetitionHorse)horse1.clone();  
horse2.setCompetitionHorseName("Thunder");  
horse2.setRider("Taylor", 1234, 25);  
String [] horse2Skills = new String[]{"Racing"};  
horse2.setCompetitionHorseSkills(horse2Skills);  
...
```

Create a Deep Copy

03_DeepCopy

Increase the herd by cloning the first horse.

```
...  
CompetitionHorse horse2 = (CompetitionHorse)horse1.clone();  
horse2.setCompetitionHorseName("Thunder");  
horse2.setRider("Taylor", 1234, 25);  
String [] horse2Skills = new String[]{"Racing"};  
horse2.setCompetitionHorseSkills(horse2Skills);  
...
```

```
Original Horse1  
Name: Blaze  
Skills: [Jumping, Dressage]  
Rider: Alex
```

```
Horse1  
Name: Blaze  
Skills: [Jumping, Dressage]  
Rider: Alex  
Horse2  
Name: Thunder  
Skills: [Racing]  
Rider: Taylor
```

Copy Constructors

Cloning is easy, only a few lines of code but requires clone() methods in all held objects

- No control over object construction and initialization

Create an object using another object

No default copy constructor like C++

Remember to include deep copies for non-primitive types

Create a Copy Constructor

```
...  
public CompetitionHorse(CompetitionHorse origCompetitionHorse){  
    this.animalSpecies = origCompetitionHorse.animalSpecies;  
    this.animalFamily = origCompetitionHorse.animalFamily;  
    this.animalPhylum = origCompetitionHorse.animalPhylum;  
    this.animalName = origCompetitionHorse.animalName;  
    this.skills = origCompetitionHorse.skills;  
    this.animalRider = new Rider(origCompetitionHorse.animalRider);  
    //Need another copy constructor!  
... }  
}
```


Exercise 15.4

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Object reference variables

Call by value vs. object reference

Shallow copies vs. deep copies

Use of Java clone() method

Copy constructors

Summary: Skills

Creating a shallow copy

Creating a deep copy

Defining and implementing clone()

Writing a copy constructor

Additional Information

Related reading

- Volume 1, Chapter 6.1.9: Object Cloning